FASE 1 - DOCUMENTACIÓN Y DISEÑO TÉCNICO

Curso de Especialización en IA y Big Data
Curso académico 2025/2026

Alumnos:

Cristian Cantero López (cristiancanterolopez@gmail.com)

Èric Garcia Dalmases (ericgada05@gmail.com)

Claudia Tello Calles (tellocallesclaudia@gmail.com)

Jesús García Quesada (jesusgarcia.301004@gmail.com)

Docente:

José Miguel Ruiz Guevara

ÍNDICE

1. TÍTULO DEL PROYECTO + INTEGRANTES DEL EQUIPO..... 2

2. RESUMEN EJECUTIVO.....2

3. DESCRIPCIÓN FUNCIONAL DEL SISTEMA..... 3

4. ALCANCE DEFINITIVO (IN / OUT)..... 4

5. ARQUITECTURA DEL SISTEMA (ALTO NIVEL).....5

6. TECNOLOGÍAS SELECCIONADAS.....6

7. MODELO DE DATOS.....7

8. DATOS.....9

9. ÉTICA, LEGALIDAD Y ACCESIBILIDAD..... 11

10. BACKLOG INICIAL..... 12

11. PLAN DE TRABAJO.....13

12. ESTRATEGIA DE DESARROLLO..... 14

13. RIESGOS TÉCNICOS ACTUALIZADOS..... 15

Enlace al repositorio de GitHub:
https://github.com/TFM-Master-en-IA-y-Big-Data/proyecto-tfm

1. TÍTULO DEL PROYECTO + INTEGRANTES DEL EQUIPO

CryptoPredict

Integrantes y roles principales

- **Claudia – Data Engineering:** ingesta de datos desde APIs, procesamiento con Spark, limpieza, almacenamiento estructurado y análisis exploratorio (EDA).
- **Cristian – Machine Learning:** construcción del dataset supervisado, entrenamiento y evaluación de modelos de clasificación, validación temporal y selección del modelo final.
- **Jesús – Platform:** desarrollo de la API con FastAPI, integración del modelo para inferencia y preparación del entorno de despliegue, además del desarrollo del frontend.
- **Eric – Business Intelligence:** diseño del dashboard en Power BI, definición de KPIs y visualización de métricas clave y comparativas.

2. RESUMEN EJECUTIVO

El mercado de criptomonedas se caracteriza por su alta volatilidad y la dispersión de datos, lo que dificulta la toma de decisiones informadas. Los inversores suelen carecer de herramientas que integren el análisis de grandes volúmenes de datos históricos con estimaciones cuantitativas del riesgo.

Para abordar este problema, se propone *CryptoPredict*, un sistema end-to-end que automatiza la ingesta de datos desde APIs públicas, su procesamiento distribuido con Apache Spark y la generación de indicadores técnicos mediante técnicas de feature engineering. Sobre estos datos se entrenan modelos de Machine Learning de clasificación binaria, capaces de predecir la probabilidad de subida o bajada de una criptomoneda. Las predicciones se exponen mediante una API REST (FastAPI) y se visualizan a través de una interfaz web y dashboards en Power BI.

El valor del proyecto reside en su arquitectura modular, escalable y reproducible, que transforma datos en información útil. Su principal aportación es la capacidad de ofrecer estimaciones probabilísticas de tendencias y riesgo, permitiendo a usuarios y analistas tomar decisiones basadas en datos y modelos predictivos.

3. DESCRIPCIÓN FUNCIONAL DEL SISTEMA

Se trata de una aplicación orientada al análisis y predicción del comportamiento de criptomonedas mediante técnicas de Big Data y Machine Learning. El sistema integra datos históricos reales del mercado, los procesa mediante pipelines automatizados y genera

predicciones probabilísticas que permiten estimar la tendencia a corto plazo y el nivel de riesgo asociado a cada activo.

Flujo general del sistema

El sistema sigue un flujo end-to-end:

- **Ingesta de datos** desde APIs externas de criptomonedas.
- **Procesamiento y almacenamiento** en el Data Lake mediante pipelines ETL con Spark.
- **Entrenamiento y reentrenamiento** del modelo ML a partir de los datos históricos y actualizados.
- **Exposición de predicciones** a través de una API REST.
- **Consumo por parte del usuario**, mediante una interfaz web y dashboards analíticos.

Casos de uso principales

- Consulta de información histórica y análisis de evolución y volatilidad.
- Solicitud de predicciones probabilísticas de subida/bajada.
- Evaluación del riesgo mediante métricas técnicas.
- Comparativa entre criptomonedas mediante dashboards.
- Seguimiento personalizado de activos favoritos.
- Uso académico para análisis de datos y modelos ML.

Capacidades del sistema

- Procesar y almacenar grandes volúmenes de datos históricos.
- Generar indicadores técnicos y datasets supervisados.
- Entrenar modelos de clasificación y producir predicciones probabilísticas.
- Visualizar resultados de forma clara e interactiva.

Limitaciones del sistema

- No predice precios exactos ni rangos futuros.
- No ejecuta trading automático.
- No ofrece asesoramiento financiero personalizado.
- No analiza criptomonedas fuera del alcance definido.

4. ALCANCE DEFINITIVO (IN / OUT)

IN:

El proyecto se centrará en cinco criptomonedas representativas del mercado, seleccionadas por capitalización, volatilidad y relevancia de negocio, para permitir análisis comparativos y entrenamiento de modelos generalizables. Las funcionalidades mínimas incluyen:

- **Ingesta y procesamiento de datos históricos** desde APIs públicas, limpieza y generación de series temporales con Apache Spark (por ejemplo, ventanas de 30 días).
- **Desarrollo de features y métricas técnicas**, capturando patrones de comportamiento y riesgo. **Construcción de un dataset supervisado** listo para Machine Learning.
- **Modelo binario MVP** para predecir subida o bajada a corto plazo de cada criptomoneda, evaluado mediante métricas estándar.
- **Exposición de predicciones mediante API REST**, consumible por la interfaz web.
- **Dashboard histórico y comparativo**, visualizando evolución, volatilidad y métricas técnicas de cada criptomoneda, facilitando análisis exploratorio.
- **Frontend multiplataforma**, que permite consultar cualquier criptomoneda incluida, calculando automáticamente features y mostrando predicciones interactivamente.
- **Arquitectura modular y reproducible**, orientada a Big Data y Machine Learning.

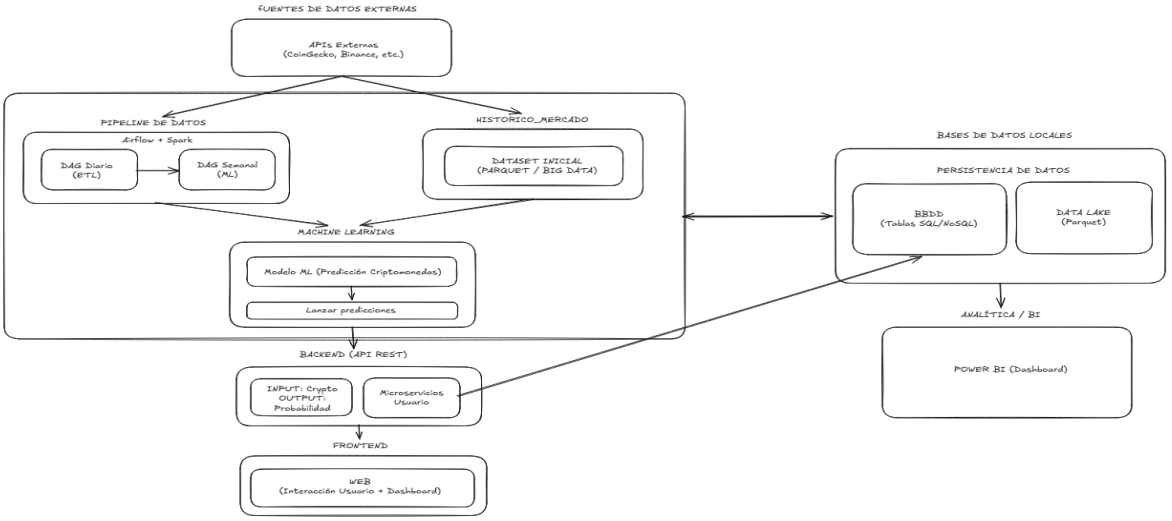
***Funcionalidad opcional**:*

- Modelo de ML adicional para caídas severas (>10%), solo si el tiempo disponible lo permite.

OUT:

- Predicción exacta de precios futuros o rangos de precio.
- Sistemas de trading automático o ejecución de órdenes.
- Recomendaciones financieras personalizadas.
- Análisis de criptomonedas minoritarias con bajo volumen o liquidez.
- Modelos financieros avanzados fuera del alcance académico del proyecto.
- Modelos de riesgo ultra-avanzados no contemplados en las funcionalidades IN.
- Uso del sistema como herramienta profesional de asesoramiento financiero.

5. ARQUITECTURA DEL SISTEMA (ALTO NIVEL)



El sistema se ha diseñado siguiendo una arquitectura modular que integra procesamiento de datos, modelado de Machine Learning, servicios backend y visualización, permitiendo un flujo completo desde la ingesta de datos hasta la exposición de predicciones.

El **frontend** consiste en una interfaz web interactiva que permite al usuario consultar criptomonedas, visualizar su evolución y obtener predicciones probabilísticas de subida o bajada junto con métricas de riesgo. Está orientado a facilitar la interpretación de los resultados de forma clara y accesible. El **backend**, implementado mediante una API REST, actúa como intermediario entre el frontend, el modelo de Machine Learning y los datos procesados, encargándose de generar las features necesarias en tiempo real y devolver las predicciones.

La **base de datos** sigue un enfoque híbrido. Por un lado, se dispone de almacenamiento transaccional para la gestión de usuarios, criptomonedas, favoritos y predicciones. Por otro, se utiliza un Data Lake en formato Parquet donde se almacenan los datos históricos procesados con Spark, que sirven como base para el análisis y el entrenamiento del modelo.

El componente de **Machine Learning** incluye el desarrollo, entrenamiento y mantenimiento de modelos de clasificación, siguiendo un enfoque de entrenamiento inicial, reentrenamiento incremental y reentrenamiento completo periódico, con evaluación continua mediante métricas estándar.

El **pipeline de datos**, automatizado con Airflow, orquesta un DAG diario de ingesta, limpieza y generación de features, y un DAG periódico de reentrenamiento del modelo y actualización de resultados. Finalmente, los **dashboards** en Power BI permiten visualizar el comportamiento histórico, métricas de riesgo y rendimiento del modelo, proporcionando una visión completa del sistema.

El sistema puede incluir **servicios adicionales** como control de versiones mediante GitHub. Estos elementos refuerzan la calidad técnica del proyecto y lo alinean con prácticas reales de desarrollo en entornos profesionales.

6. TECNOLOGÍAS SELECCIONADAS

Lenguajes

- **Python:** Elegido por su ecosistema en Machine Learning y procesamiento de datos (Pandas, PySpark). Alternativa: R (descartado por menor integración con el stack backend).
- **JavaScript:** Para el desarrollo del frontend web y consumo de la API. Alternativa: TypeScript (no prioritario en fase inicial).

Frameworks

- **FastAPI:** Alta velocidad, tipado y facilidad para exponer modelos ML mediante APIs REST. Alternativa: Flask (menos estructurado) / Django (más pesado).

- **Apache Airflow:** Orquestación de pipelines de datos mediante DAGs y automatización de tareas ETL/ML. Alternativa: Cron (menos escalable).

Librerías principales

- **PySpark:** Procesamiento distribuido de grandes volúmenes de datos. Alternativa: Pandas (limitado para Big Data).
- **Scikit-Learn:** Entrenamiento de modelos de ML supervisados.
- **Pandas:** Manipulación y análisis de datos en fases iniciales y exploratorias. Alternativa: Polars.
- **Seaborn / Matplotlib:** Visualización de análisis exploratorio y gráficos estadísticos.

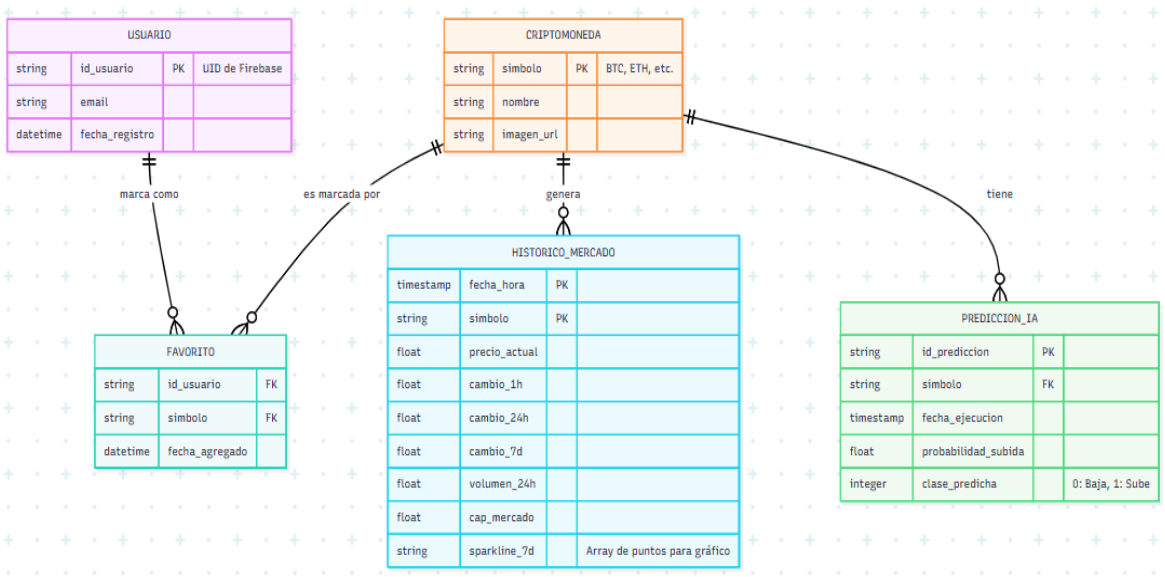
Tipo de base de datos

- **Data Lake (Parquet):** Almacenamiento eficiente y escalable de datos históricos. Alternativa: Data Warehouse tradicional.
- **Bases de Datos SQL/NoSQL (Tablas de referencia + Firebase):** Gestión de usuarios, criptomonedas y favoritos de forma estructurada.

Herramientas auxiliares

- **Power BI:** Visualización de métricas y análisis de datos en dashboards. Alternativa: Tableau / Grafana.
- **GitHub:** Control de versiones y trabajo colaborativo. Alternativa: GitLab.

7. MODELO DE DATOS



Explicación de tablas/colecciones

- **USUARIOS (Gestionado vía Firebase Auth)**: Almacena la identidad de los alumnos/inversores. Al usar Firebase, la seguridad de la contraseña está delegada, almacenando nosotros solo el UID y el email.
- **CRIPTOMONEDAS:** Tabla maestra que actúa como catálogo de los activos que el sistema es capaz de monitorizar (las "Top 5" seleccionadas).
- **FAVORITOS:** Tabla de unión que permite la persistencia de las preferencias. Un usuario puede seguir N monedas y una moneda puede ser seguida por N usuarios.
- **HISTORICO_MERCADO (Capa Big Data - Parquet):** Es la tabla más voluminosa. Aquí Spark guarda los snapshots de mercado que permitirán pintar la tabla inicial (con los % de 1h, 24h, 7d) y los gráficos de tendencia.
- **PREDICCIONES_IA:** Tabla donde el modelo de Machine Learning vuelca sus resultados tras procesar los datos con Python.

Campos principales

- **Claves Primarias (PK):** *id_usuario*, *simbolo* (único para cada moneda), y la combinación (*fecha_hora*, *simbolo*) para los datos históricos, asegurando que no haya duplicados para un mismo momento temporal.
- **Claves Foráneas (FK):** *simbolo* actúa como el eje central que conecta los precios reales con las predicciones de la IA y los favoritos del usuario.
- **Campos de Tendencia:** Los campos *cambio_1h*, *24h* y *7d* son calculados por Spark comparando el precio actual con registros pasados en el Data Lake.

Relaciones

El modelo se articula en torno a tres ejes relacionales principales:

- **Relación Usuario - Favoritos (1:N):** Cada usuario registrado en el sistema mediante Firebase Auth puede marcar múltiples criptomonedas como "Favoritas". Esta relación permite personalizar la vista inicial del dashboard para cada perfil.
- **Relación Criptomoneda - Histórico (1:N):** Cada activo (ej. Bitcoin) tiene asociados miles de registros temporales (snapshots). Estos registros contienen los precios y porcentajes de variación (1h, 24h, 7d) necesarios para generar los gráficos de tendencia (sparklines).
- **Relación Criptomoneda - Predicción (1:1/N):** Por cada ventana temporal de ejecución del modelo de Machine Learning, se genera una predicción de riesgo y tendencia para cada criptomoneda soportada por el sistema.

Claves

Para garantizar la integridad de los datos y la rapidez en las consultas de la API, se han definido las siguientes claves:

Claves Primarias (Primary Keys):

- *id_usuario*: Identificador único (UID) proporcionado por Firebase.
- *simbolo_cripto*: Código único del activo (ej: "BTC", "ETH", "SOL"). Se usa como clave natural.
- *id_registro_temporal*: Clave compuesta por simbolo_cripto + timestamp, que identifica de forma única el precio de una moneda en un momento exacto.

Claves Foráneas (Foreign Keys):

- *simbolo_cripto* (en tablas de Favoritos y Predicciones): Vincula los resultados de la IA y las preferencias del usuario con el maestro de criptomonedas.
- *id_usuario* (en tabla de Favoritos): Vincula la selección de monedas con el perfil de usuario autenticado.

Diccionario de variables

El dataset utilizado para entrenar el modelo de clasificación binaria (subida/bajada) y estimación de riesgo se compone de las siguientes variables extraídas y calculadas mediante Apache Spark:

Variable	**Tipo**	**Descripción**	**Relevancia en el Modelo**
timestamp	Datetime	Marca temporal que indica el momento en el que se registra cada dato.	Permite ordenar los datos en el tiempo.
price_usd	Float	Precio de cierre actual en dólares.	Base para el cálculo de retornos.
change_1h_pct	Float	Variación porcentual del precio en la última hora.	Captura la micro-tendencia inmediata.
change_24h_pct	Float	Variación porcentual del precio en las últimas 24 horas.	Indicador de tendencia diaria.
change_7d_pct	Float	Variación porcentual del precio en los últimos 7 días.	Captura el sentimiento de mercado semanal.

volume_24h	Float	Volumen total transaccionado en el último día.	Indica la fuerza o liquidez del movimiento.
market_cap	Float	Capitalización total del mercado del activo.	Define la estabilidad relativa del activo.
rsi_14	Float	Relative Strength Index (Indicador Técnico).	Identifica condiciones de sobrecompra/venta.
volatility_30d	Float	Desviación estándar de los precios en 30 días.	Fundamental para la predicción de caídas severas.
target	Int	Variable Objetivo (Label): 1 si sube, 0 si baja, según un umbral de variación porcentual definido.	Clase que el modelo debe predecir.

8. DATOS

El sistema se alimenta de datos históricos reales del mercado de criptomonedas, obtenidos a través de APIs públicas especializadas en información financiera.

Fuentes

Las principales fuentes de datos serán:

- CoinGecko API.
- Binance API.

Estas APIs proporcionan información actualizada sobre precios, volumen de transacciones y capitalización de mercado.

Licencia

Los datos utilizados provienen de fuentes públicas con licencias de uso gratuito para fines académicos y no comerciales, respetando en todo momento los términos de uso de cada proveedor.

Tipo de datos

El sistema trabaja principalmente con datos estructurados en formato de series temporales, incluyendo, entre otros: precio de cierre, volumen de transacciones, capitalización de mercado e indicadores derivados (RSI, volatilidad, retornos).

Volumen estimado

- 5 criptomonedas seleccionadas.
- Histórico de entre 2 y 3 años.
- Frecuencia diaria.

Esto supone aproximadamente:

- 5.000–10.000 registros base.
- Dataset ampliado tras feature engineering con aproximadamente 10 variables.

Frecuencia de actualización

- Ingesta inicial completa del histórico.
- Actualización periódica: diaria mediante DAG de Airflow y con una posible ampliación a frecuencia horaria.

Calidad esperada

La calidad de los datos es alta, al provenir de fuentes consolidadas. No obstante, pueden aparecer:

- Valores faltantes puntuales.
- Picos de volatilidad.

Riesgos asociados a los datos

- **Datos faltantes:** mitigados mediante imputación o eliminación controlada.
- **Ruido en los datos:** suavizado mediante ventanas móviles.
- **Sesgos temporales:** controlados mediante validación temporal.
- **Inconsistencias entre fuentes:** uso de una API principal para estandarización.

Naturaleza de los datos

El sistema trabaja exclusivamente con datos reales procedentes de fuentes externas, sin utilizar datos simulados ni mockeados en ninguna fase del proyecto.

9. ÉTICA, LEGALIDAD Y ACCESIBILIDAD

Datos personales y privacidad

El sistema recolecta únicamente el correo electrónico y la contraseña del usuario para la creación de perfiles y la gestión de criptomonedas favoritas. No se solicitan datos sensibles como nombre real, dirección, teléfono o información bancaria.

Cumplimiento normativo (RGPD)

Se garantiza la seguridad delegando la autenticación en Firebase, que aplica cifrado y estándares internacionales. El tratamiento de datos se basa en el consentimiento del usuario y en la prestación del servicio. Además, el sistema permite ejercer derechos de acceso, rectificación y supresión (eliminación de cuenta). Se prioriza el almacenamiento de datos dentro de la Unión Europea.

Riesgos de sesgo

El modelo puede verse afectado por sesgos como supervivencia (uso de criptos actuales), look-ahead (uso de información futura), ciclo temporal (datos de mercado alcista) y frecuencia (uso de datos diarios que ocultan volatilidad intradía).

Medidas de mitigación

Se aplican técnicas como validación temporal, auditorías periódicas del modelo y uso de buenas prácticas en la preparación de datos. Además, se implementan medidas de seguridad como hashing de contraseñas y avisos de transparencia sobre el carácter no financiero de las predicciones.

Accesibilidad

El sistema incluye formularios accesibles, mensajes de error claros y navegación mediante teclado, facilitando su uso a personas con distintas capacidades.

10. BACKLOG INICIAL

El backlog inicial se compone de las siguientes historias de usuario, que cubren las funcionalidades principales del sistema:

1. Ingesta de datos

Como sistema, quiero obtener datos históricos desde APIs públicas para construir el dataset inicial. **Criterios de aceptación:**

- Conexión correcta con la API.
- Datos almacenados en formato Parquet.
- Ausencia de duplicados.

2. Limpieza de datos

Como sistema, quiero limpiar los datos para garantizar su calidad antes del procesamiento. **Criterios de aceptación:**

- Eliminación de valores nulos o inconsistentes.
- Dataset listo para análisis.

3. Feature engineering

Como sistema, quiero generar indicadores técnicos para enriquecer el dataset. **Criterios de aceptación:**

- Cálculo correcto de RSI, volatilidad y retornos.
- Nuevas variables añadidas al dataset.

4. Construcción del dataset supervisado

Como sistema, quiero generar la variable objetivo para entrenar el modelo. **Criterios de aceptación:**

- Variable target correctamente definida.
- Dataset preparado para Machine Learning.

5. Entrenamiento del modelo

Como sistema, quiero entrenar un modelo de clasificación binaria. **Criterios de aceptación:**

- Modelo entrenado correctamente.
- Métricas calculadas (Accuracy, F1-score, ROC-AUC).

6. Evaluación del modelo

Como sistema, quiero validar el rendimiento del modelo. **Criterios de aceptación:**

- Uso de validación temporal
- Resultados documentados

7. Desarrollo de la API

Como sistema, quiero exponer las predicciones mediante una API REST. **Criterios de aceptación:**

- Endpoint funcional (/predict).
- Respuesta en formato JSON.

8. Integración modelo–API

Como sistema, quiero conectar el modelo con la API para inferencia en tiempo real. **Criterios de aceptación:**

- Predicciones generadas correctamente.
- Tiempo de respuesta adecuado.

9. Desarrollo del frontend

Como usuario, quiero visualizar las predicciones de forma clara. **Criterios de aceptación:**

- Selector de criptomonedas funcional.
- Visualización de resultados.

10. Dashboard de Business Intelligence

Como usuario, quiero analizar datos históricos y métricas clave. **Criterios de aceptación:**

- Visualización de KPIs.
- Comparativas entre activos.

11. PLAN DE TRABAJO

El desarrollo del proyecto se organiza en un cronograma de 12 semanas, estructurado en fases progresivas que permiten la construcción incremental del sistema.

Cronograma

- Semanas 1–2: Ingesta de datos.
- Semanas 3–4: Limpieza y análisis exploratorio (EDA).
- Semanas 5–6: Feature engineering.
- Semanas 7–8: Desarrollo y entrenamiento del modelo.
- Semana 9: Desarrollo de la API.
- Semana 10: Desarrollo del frontend.
- Semana 11: Creación de dashboards.
- Semana 12: Integración final y testing.

Hitos principales

- Dataset limpio y estructurado.
- Modelo de Machine Learning entrenado.
- API funcional.
- Interfaz de usuario operativa.
- Sistema completo integrado.

Dependencias

El proyecto presenta las siguientes dependencias:

- El modelo depende de la disponibilidad del dataset limpio.
- La API depende del modelo entrenado.
- El frontend depende de la API.

Reparto de tareas

El trabajo se distribuye según los roles definidos:

- Data Engineering: pipeline de datos.
- Machine Learning: modelado y evaluación.
- Backend/Frontend: API e interfaz.
- Business Intelligence: dashboards.

Riesgos técnicos

- Dependencia de APIs externas.
- Problemas en la calidad de los datos.
- Complejidad en la integración de componentes.

12. ESTRATEGIA DE DESARROLLO

El desarrollo del proyecto seguirá buenas prácticas de ingeniería de software, garantizando calidad, trazabilidad y colaboración efectiva.

Control de versiones

Se utilizará GitHub con la siguiente estructura de ramas:

- *main*: versión estable.
- *develop*: integración de funcionalidades.
- *feature/**: desarrollo de nuevas características.

Integración de código

- Uso de Pull Requests para integrar cambios.
- Revisión de código entre miembros del equipo.
- Integración progresiva en la rama *develop*.

Validación funcional

La validación del sistema se realizará mediante:

- Pruebas manuales de la API.
- Verificación de predicciones.
- Comparación con datos históricos reales.

Testing básico

Se implementarán pruebas básicas:

- Tests unitarios para funciones clave.
- Test de endpoints mediante herramientas como POSTMAN.
- Validación de consistencia de datos.

13. RIESGOS TÉCNICOS ACTUALIZADOS

El proyecto presenta una serie de riesgos técnicos que han sido identificados junto con sus estrategias de mitigación:

1. Dependencia de APIs externas

- Impacto: Alto.
- Probabilidad: Media.
- Mitigación: Uso de almacenamiento local y caché de datos.

2. Datos incompletos o inconsistentes

- Impacto: Medio.
- Probabilidad: Media.
- Mitigación: Procesos de limpieza y validación.

3. Sobreajuste del modelo (overfitting)

- Impacto: Alto.
- Probabilidad: Alta.
- Mitigación: Validación temporal y regularización.

4. Bajo rendimiento del modelo

- Impacto: Alto.
- Probabilidad: Media.
- Mitigación: Evaluación de múltiples algoritmos.

5. Problemas de integración

- Impacto: Medio.
- Probabilidad: Media.

- Mitigación: Integración progresiva y testing continuo.

6. Latencia en la API

- Impacto: Medio.
- Probabilidad: Baja.
- Mitigación: Optimización del modelo y uso de caché.

7. Limitaciones de tiempo

- Impacto: Alto.
- Probabilidad: Media.
- Mitigación: Priorización del MVP y funcionalidades clave.